

DEVOPS DEPLOYMENT GUIDE

DevOps Deployment Guide

dealing cluster

CLUSTER dealing cluster

AUDIENCE App developers shipping to dealing

CLASSIFICATION Internal — keep current as the platform evolves

Contents

01	TL;DR — your first app in 5 steps	3
02	1. What's automatic vs what's your job	3
03	2. Namespace setup	4
04	3. The standard app — full working example	5
05	4. Deploying via ArgoCD	7
06	5. Image pulls	8
07	6. Secrets	9
08	7. NetworkPolicy — when (rarely) you need one	9
09	8. Metrics — automatic scraping	11
10	9. Logs — automatic shipping	11
11	10. TLS for your Ingress	12
12	11. Pre-flight checklist for each new deployment	12
13	12. After deployment — sanity checks	13
14	13. Common gotchas	13
15	14. Where to go next	14
16	15. Who to ask	14

One-page-feel checklist + working examples for shipping an app to this cluster.

If you're the platform/ops team, read the `cluster-handbook` instead. This guide is for the **app team about to deploy a new workload**.

TL;DR — your first app in 5 steps

1. Pick a namespace name (e.g. `webtrader`). Create the namespace with PSS label.
2. Write a Deployment that **declares resource requests + limits**, uses non-root, and points at your image.
3. Write a Service (`ClusterIP`) + Ingress (`ingressClassName: cilium`) for HTTPS.
4. Push the YAML to Git → ArgoCD picks it up → app runs.
5. Open `https://yourapp.dealing.internal` in your browser.

The cluster handles DNS, Prometheus scraping, TLS, public-internet egress, and pod-to-pod encryption automatically. You don't write a NetworkPolicy unless you need *tighter* than the baseline.

1. What's automatic vs what's your job

What the cluster does for you	What you have to do
Your pod can resolve DNS (in-cluster + external)	Use real hostnames in your config — they'll resolve
Your pod can reach the kube-apiserver	If your app uses K8s API, just use the in-cluster URL
Your pod can reach any other pod in the cluster	No per-pod allowlist needed
Your pod can reach the public internet (Slack, GitHub, AWS, ...)	Just call the URL
Your pod can reach cross-VLAN internal services (switch ACL allows)	Use the real IP/hostname
External users can reach you via Ingress	Define an Ingress resource
Prometheus auto-scrapes your <code>/metrics</code>	Define a ServiceMonitor (or annotations)
TLS certs for your Ingress	Add a <code>cert-manager.io/cluster-issuer: cluster-ca-issuer</code> annotation
Pod logs → Loki	Just write to stdout / stderr (containerd + Alloy do the rest)

What the cluster does for you	What you have to do
Pod-to-pod traffic encrypted across nodes	WireGuard, automatic, no setup
Image pull from public registries	Just use the image reference (<code>ghcr.io/...</code> , <code>docker.io/...</code>)
Resource requests + limits on your pods	YES, on every container
PSS-compliant securityContext (non-root, no privilege escalation, drop ALL caps)	YES, on every container if you use <code>baseline</code> or <code>restricted</code> PSS
Pulling from a private registry	YES, create an <code>imagePullSecret</code> first
TLS / authn at the app layer	YES — Cilium WireGuard protects in-transit between nodes, but app should still terminate TLS for user-facing traffic
Defining the Ingress hostname	YES — pick <code><yourapp>.dealing.internal</code>
A NetworkPolicy	Only if you need TIGHTER than the baseline (see §7)

2. Namespace setup

```

apiVersion: v1
kind: Namespace
metadata:
  name: webtrader
  labels:
    # Pod Security Standards – pick one:
    pod-security.kubernetes.io/enforce: baseline    # most apps
    # pod-security.kubernetes.io/enforce: restricted # production-grade apps (preferred)
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted

```

Which PSS profile do I use?

Profile	When	What your pods need
<code>baseline</code>	Most apps. Default safe choice.	No <code>hostNetwork: true</code> , no <code>privileged: true</code> , no <code>hostPID/</code> <code>hostIPC</code> , no <code>hostPath</code> volumes (mostly).
<code>restricted</code>	Production data-handling apps.	Above PLUS: <code>runAsNonRoot: true</code> , <code>allowPrivilegeEscalation: false</code> , drop all caps, <code>seccompProfile.type: RuntimeDefault</code> .

You can always start with `baseline` and tighten to `restricted` later.

3. The standard app — full working example

This is the canonical shape. Copy-paste, replace `webtrader` with your app name.

```

---
# Deployment – your app
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webtrader
  namespace: webtrader
  labels:
    app.kubernetes.io/name: webtrader
    app.kubernetes.io/component: api
spec:
  replicas: 2
  selector:
    matchLabels:
      app.kubernetes.io/name: webtrader
  template:
    metadata:
      labels:
        app.kubernetes.io/name: webtrader
    spec:
      securityContext:                                     # PSS restricted boilerplate
        runAsNonRoot: true
        runAsUser: 1000
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: webtrader
          image: ghcr.io/your-org/webtrader:1.4.2          # pinned tag, NOT :latest
          imagePullPolicy: IfNotPresent
          securityContext:
            allowPrivilegeEscalation: false
            runAsNonRoot: true
            runAsUser: 1000
            capabilities:
              drop: ["ALL"]
            seccompProfile:
              type: RuntimeDefault
          ports:
            - name: http
              containerPort: 8080
            - name: metrics
              containerPort: 9090
          # REQUIRED. Without these, scheduler can't place you reliably + kubectl top is
          # blind.
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 512Mi
          readinessProbe:
            httpGet: {path: /healthz, port: http}
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet: {path: /healthz, port: http}
            initialDelaySeconds: 30
            periodSeconds: 30
---
# Service – what other pods + Ingress route to
apiVersion: v1
kind: Service
metadata:
  name: webtrader
  namespace: webtrader
  labels:

```

```

    app.kubernetes.io/name: webtrader
spec:
  type: ClusterIP
  selector:
    app.kubernetes.io/name: webtrader
  ports:
    - name: http
      port: 80
      targetPort: http
    - name: metrics
      port: 9090
      targetPort: metrics
---
# Ingress – how external users reach you (via Cilium IngressController)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webtrader
  namespace: webtrader
  annotations:
    cert-manager.io/cluster-issuer: cluster-ca-issuer
spec:
  ingressClassName: cilium
  rules:
    - host: webtrader.dealing.internal
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: webtrader
                port:
                  number: 80
  tls:
    - hosts:
        - webtrader.dealing.internal
      secretName: webtrader-tls # cert-manager will create this

```

Push this YAML to a Git repo, then add it to ArgoCD (next section).

4. Deploying via ArgoCD

ArgoCD is the deploy mechanism on this cluster. **Don't** `kubectl apply` directly to production-facing namespaces — go through ArgoCD so the state is reconciled from Git.

ArgoCD URL + login

- URL: `https://argocd.dealing.internal`
- Initial admin password: in the secret `argocd/argocd-initial-admin-secret` (check with platform team for the current value, or get the initial one with `kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d`)
- Change the admin password after first login. Set up per-user SSO if your org uses one.

Register your app as an ArgoCD Application

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: webtrader
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-org/webtrader-manifests
    targetRevision: main
    path: deploy/dealing
  destination:
    server: https://kubernetes.default.svc
    namespace: webtrader
  syncPolicy:
    automated:
      prune: true      # ArgoCD deletes resources removed from Git
      selfHeal: true   # ArgoCD reconciles manual `kubectl edit` drift
    syncOptions:
      - CreateNamespace=true  # auto-create the namespace if missing
```

Push this to the ArgoCD git repo (or apply it directly with `kubectl -n argocd apply -f`). ArgoCD takes it from there.

5. Image pulls

Public registries (no setup needed)

Just reference them. `containerd` on each node pulls directly:

```
image: ghcr.io/example/webtrader:1.4.2
image: docker.io/library/nginx:1.27
image: quay.io/prometheus/prometheus:v3.8.0
image: registry.k8s.io/kube-state-metrics/kube-state-metrics:v2.17.0
```

Private registry (need a pull secret)

```
kubectl -n webtrader create secret docker-registry myregistry-creds \
  --docker-server=registry.mycorp.com \
  --docker-username='<user>' \
  --docker-password='<password>' \
  --docker-email='ops@mycorp.com'
```

Then in your Deployment:

```
spec:
  template:
    spec:
      imagePullSecrets:
        - name: myregistry-creds
      containers:
        - image: registry.mycorp.com/webtrader:1.4.2
```

Never use `:latest`

Always pin to a specific tag (or even better, an image digest `@sha256:...`). Floating tags break reproducibility and security audits.

6. Secrets

Avoid committing secrets to Git. Two paths:

Native K8s Secret (acceptable for low-sensitivity, rotation-friendly)

```
kubectl -n webtrader create secret generic webtrader-config \
  --from-literal=DATABASE_PASSWORD='<value>' \
  --from-literal=API_KEY='<value>'
```

Reference in Deployment:

```
env:
  - name: DATABASE_PASSWORD
    valueFrom: {secretKeyRef: {name: webtrader-config, key: DATABASE_PASSWORD}}
```

`external-secrets` (preferred for sensitive secrets)

`external-secrets` is deployed but **not configured** on this cluster yet. If you need it, ask platform team to set up a `ClusterSecretStore` pointing at your org's secret backend (Vault / AWS SM / Azure KV). Then you declare an `ExternalSecret` CR that names the secret to sync.

7. NetworkPolicy — when (rarely) you need one

You **don't need one** for most apps. The cluster-wide CCNPs already grant: - Egress: in-cluster, DNS, kube-apiserver, public internet (subject to switch ACLs) - Ingress: Ingress controller, Prometheus, in-namespace pod-to-pod, kube-system tools

You **do need one** if your app should be tighter than baseline. Common cases:

Lock egress to a specific external service only

```
# Payments app: only Stripe API allowed
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: payments-strict-egress
  namespace: payments
spec:
  endpointSelector:
    matchLabels:
      app.kubernetes.io/name: payments
  enableDefaultDeny:
    egress: true # default-deny for this app
  egress:
    - toFQDNs:
        - matchPattern: "*.stripe.com"
      toPorts:
        - ports: [{port: "443", protocol: TCP}]
    # Don't forget DNS – otherwise the app can't even resolve stripe.com
    - toEndpoints:
        - matchLabels:
            k8s:io.kubernetes.pod.namespace: kube-system
            k8s-app: kube-dns
      toPorts:
        - ports:
            - port: "53"
              protocol: UDP
            - port: "53"
              protocol: TCP
    # K8s API (if your app calls it)
    - toEntities: [kube-apiserver]
```

Lock ingress to only specific other namespaces

```
# Internal admin tool: accept ingress only from the bastion namespace
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: admin-tool-ingress-only-from-bastion
  namespace: admin-tool
spec:
  endpointSelector:
    matchLabels:
      app.kubernetes.io/name: admin-tool
  enableDefaultDeny:
    ingress: true
  ingress:
    - fromEndpoints:
        - matchLabels:
            k8s:io.kubernetes.pod.namespace: bastion
```

For any NetworkPolicy / CNP / CCNP, read [policy-reviewer.md](#) first. Common mistakes (empty selector, forgetting DNS in egress allowlist, blocking yourself from kube-apiserver) are documented there.

8. Metrics — automatic scraping

Prometheus auto-scrapes any pod with a `ServiceMonitor` (or a Service with the right annotation if you use legacy patterns). Drop this alongside your Deployment + Service:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: webtrader
  namespace: webtrader
  labels:
    release: kube-prometheus-stack      # IMPORTANT — picked up by our Prometheus selector
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: webtrader
  endpoints:
    - port: metrics                    # the port name from your Service
      interval: 30s
      path: /metrics
```

After ~30 seconds, your metrics are queryable in Mimir (visible in Grafana) with the label

```
cluster_name="dealing".
```

To verify your scrape target shows up:

```
kubectl -n monitoring exec prometheus-kube-prometheus-stack-prometheus-0 -c prometheus -- \
  wget -qO- 'http://localhost:9090/api/v1/targets?state=active' | python3 -c "import
  sys,json; [print(t['scrapeUrl'], t['health']) for t in json.load(sys.stdin)['data']
  ['activeTargets']] if 'webtrader' in t.get('scrapeUrl',')"]"
```

9. Logs — automatic shipping

Just write to `stdout` / `stderr`. Don't open log files inside the container — they won't be captured.

```
# Python example
import sys, json
print(json.dumps({"event": "user_login", "user_id": 42}), flush=True)
```

Grafana Alloy on each node tails `/var/log/pods/**/*.*.log`, ships to central Loki. Query in Grafana → Explore → Loki:

```
{namespace="webtrader"} |= "error"          # all error lines
{namespace="webtrader", container="webtrader"} # specific container
```

10. TLS for your Ingress

Done by cert-manager via the `cert-manager.io/cluster-issuer: cluster-ca-issuer` annotation on your Ingress (see §3 example). cert-manager creates the Secret named in `spec.tls[].secretName` automatically.

Available issuers: - `cluster-ca-issuer` — internal CA. Use for `*.dealing.internal`, `*.cluster.internal`. - `selfsigned-issuer` — bootstrap only. Don't use for real apps. - `letsencrypt-prod` / `letsencrypt-staging` — currently NotReady. Don't use until platform team fixes ACME solver setup.

The first time a user visits `https://webtrader.dealing.internal`, their browser may warn that the cert is from an internal CA. To avoid this, the cluster CA's root cert should be added to their browser/OS trust store (handled by IT for corporate machines).

11. Pre-flight checklist for each new deployment

Before merging the PR / clicking "sync" in ArgoCD:

- ❑ Image tag is pinned (not `:latest`)
- ❑ Container has resource requests AND limits
- ❑ Container has securityContext: non-root + drop ALL caps + seccompProfile (for restricted PSS)
- ❑ Container has readiness + liveness probes
- ❑ Service is ClusterIP (not NodePort – use Ingress instead)
- ❑ Ingress has cert-manager annotation (TLS will be issued automatically)
- ❑ Ingress host follows pattern: `<yourapp>.dealing.internal`
- ❑ ServiceMonitor exists if you expose `/metrics`
- ❑ Secrets are NOT in the YAML – use Secret + secretKeyRef or ExternalSecret
- ❑ If you wrote a NetworkPolicy, you've cross-checked it against `policy-reviewer.md`

12. After deployment — sanity checks

```
# 1. Pod is running
kubectl -n webtrader get pod -l app.kubernetes.io/name=webtrader

# 2. Endpoints are populated
kubectl -n webtrader get endpoints webtrader

# 3. Ingress has the LB IP
kubectl -n webtrader get ingress webtrader -o wide

# 4. cert-manager issued the cert
kubectl -n webtrader get certificate
# Should show READY=True after a minute. If not:
kubectl -n webtrader describe certificate webtrader-tls

# 5. Try the URL
curl -k https://webtrader.dealing.internal/healthz

# 6. Metrics are flowing (after ~1 min)
# In Grafana, query: up{job="webtrader", cluster_name="dealing"}

# 7. Logs are flowing (after ~30s)
# In Grafana Explore → Loki: {namespace="webtrader"}
```

13. Common gotchas

Symptom	Cause	Fix
Pod stays Pending	Resource requests too large for any node, or PVC unbound	<code>kubectl describe pod</code> — error message is explicit
Pod stays ContainerCreating	Image pull failing	<code>kubectl describe pod</code> — check pull errors, registry creds
Pod CrashLoopBackOff right after start	App crashes on startup	<code>kubectl logs <pod> --previous</code>
Pod runs but readiness probe fails	App not listening on the configured port, or <code>/healthz</code> not implemented	Add the endpoint or fix the port
Ingress returns 404	Host header mismatch	Verify your DNS resolves to <code>10.10.120.140</code> and Host header matches Ingress rule
Ingress returns 503	Backend Service has no Endpoints	Service selector doesn't match Pod labels — fix the selector
TLS cert never gets issued	Issuer is not Ready	<code>kubectl get clusterissuer</code> — only <code>cluster-ca-issuer</code> is Ready

Symptom	Cause	Fix
App can't reach external API	DNS or egress	Check Hubble: <code>hubble observe --from-namespace webtrader --verdict DROPPED</code>
App can't reach internal service across VLAN	Switch ACL doesn't permit that port	Ask network team to open the port
Metrics not showing in Grafana	ServiceMonitor missing <code>release: kube-prometheus-stack</code> label	Add it
Logs not showing in Loki	App writing to a file, not stdout	Make app log to stdout/stderr

14. Where to go next

Want to	Read
Understand the cluster end-to-end	<code>cluster-handbook-dealing.md</code> — start with §25 (end-to-end flows)
Write a NetworkPolicy / CNP / CCNP	<code>policy-reviewer.md</code> — read before applying
Find suggested CPU/mem requests	<code>workload-catalog-dealing.md</code> §2.14 (per-workload baseline table)
Understand a Grafana panel	<code>monitoring-playbook-dealing.md</code>
Investigate an incident with your app	<code>runbook-dealing.md</code> (ops-focused, but useful for IR-4 Pod CrashLooping)

15. Who to ask

Question	Owner
Cluster is broken / api is slow / dashboards red	Platform / on-call
Need a switch ACL changed (cross-VLAN port)	Network team
Need a new ClusterIssuer / cert-manager change	Platform
Need an ImagePullSecret to a new private registry	DevOps team can create it in their own namespace
Need a CCNP changed (cluster-wide policy)	Platform team — open a PR against <code>security/network-policies/</code>

Question	Owner
Need a Kyverno policy added / changed	Platform team — open a PR against <code>security/kyverno-policies/</code>
Need more Mimir retention / new dashboard for our app	Observability / platform

This guide is intentionally short. If you find yourself doing something not in here that other app teams will also need, add a section. PRs welcome.